

SpringBoot扩展点全攻略：让你的代码像积木一样灵活组装

夏壹 程序语言 2026年4月6日 07:00 北京

01

扩展点的核心本质，是在代码中预设标准化的**扩展插槽**。
新增功能时，只需开发对应的“插件组件”接入插槽，无需修改原有业务逻辑。

传统开发 VS 扩展点开发：核心差异对比



```
// 传统开发模式：功能耦合，改一处动全身
public class UserService {
    public void login(User user) {
        // 核心登录逻辑
        validateUser(user); // 校验用户信息
        updateLoginTime(user); // 更新登录时间

        // 新增功能直接嵌入，耦合度拉满
        sendSMS(user); // 发送登录短信
        logLoginEvent(user); // 记录登录日志
        updateLoginCount(user); // 更新登录次数
    }
}

// 扩展点开发模式：基于事件解耦，功能独立可插拔
public class UserService {
    // 注入Spring事件发布者，核心依赖抽象而非具体实现
    private ApplicationEventPublisher eventPublisher;

    public void login(User user) {
        // 只保留核心登录逻辑，聚焦单一职责
        validateUser(user);
        updateLoginTime(user);

        // 发布登录事件，不关心后续谁处理、怎么处理
        eventPublisher.publishEvent(new UserLoginEvent(user));
    }
}

// 短信通知：独立的事件处理器，可按需新增/移除
public void sendSmsOnLogin(UserLoginEvent event) {
    smsService.sendLoginNotification(event.getUser());
}

// 日志记录：独立的事件处理器，与核心逻辑解耦
public void logLoginEvent(UserLoginEvent event) {
    logService.recordLogin(event.getUser());
}
```

对比维度	传统做法	扩展点做法
------	------	-------

对比维度	传统做法	扩展点做法
代码耦合度	高，核心逻辑与扩展功能混杂	低，各功能模块独立维护
维护难度	难，改一处可能影响多个功能	易，每个组件各司其职
功能扩展	必须修改原有代码，风险高	新增组件即可，零侵入
单元测试	复杂，需Mock大量依赖	简单，单个组件独立测试
团队协作	代码冲突频发	可并行开发，互不干扰

02

SpringBoot核心扩展点：按场景精准落地

SpringBoot在应用启动、Bean生命周期、请求处理等全流程提供了扩展点，不同扩展点对应不同的业务场景，精准选择才能最大化发挥价值。

02.1 启动阶段扩展：ApplicationListener

扩展点说明： `ApplicationListener` 是SpringBoot最常用的启动扩展点，可监听应用启动全生命周期的各类事件。
其中监听 `ApplicationReadyEvent` 事件，能在应用完全启动、可接收请求后执行初始化操作。

适用场景： 缓存预热、外部服务连通性检查、定时任务启动、系统状态初始化等。



```

/**
 * 应用启动完成监听器
 * 监听ApplicationReadyEvent事件，确保应用完全启动后执行初始化
 */
public class ApplicationStartupListener implements ApplicationListener<ApplicationReadyEvent> {

    // 注入Redis操作模板
    @Resource
    private RedisTemplate<String, Object> redisTemplate;
    // 注入数据预热服务
    @Resource
    private DataWarmupService dataWarmupService;

    @Override
    public void onApplicationEvent(ApplicationReadyEvent event) {
        log.info("应用启动完成，开始执行启动后初始化任务...");

        warmupCache(); // 缓存预热
        checkExternalServices(); // 外部服务检查
        startBackgroundTasks(); // 启动后台任务

        log.info("应用启动后初始化任务执行完成");
    }

    /**
     * 缓存预热：加载热门商品数据到Redis，提升查询性能
     */
    private void warmupCache() {
        try {
            // 获取热门商品列表
            List<Product> hotProducts = dataWarmupService.getHotProducts();
            // 批量写入Redis，设置1小时过期时间
            hotProducts.forEach(product -> {
                String key = "product:" + product.getId();
                redisTemplate.opsForValue().set(key, product, Duration.ofHours(1));
            });
            log.info("缓存预热完成，预热商品数量: {}", hotProducts.size());
        } catch (Exception e) {
            log.error("缓存预热失败", e); // 异常捕获，避免影响主流程
        }
    }
}

```

```

/**
 * 检查外部服务连通性：支付、短信等核心服务
 */
private void checkExternalServices() {
    // 支付服务检查
    if (paymentService.isAvailable()) {
        log.info("支付服务连接正常");
    } else {
        log.warn("支付服务连接异常，请检查配置");
    }
    // 短信服务检查
    if (smsService.isAvailable()) {
        log.info("短信服务连接正常");
    } else {
        log.warn("短信服务连接异常，请检查配置");
    }
}

/**
 * 启动后台定时任务
 */
private void startBackgroundTasks() {
    // 此处可初始化定时任务、异步任务等
}
}

```

02.2 启动任务扩展：CommandLineRunner

扩展点说明： `CommandLineRunner` 允许应用启动完成后立即执行代码，支持接收命令行参数，可通过 `@Order` 注解控制多个Runner的执行顺序。

适用场景： 数据库初始化、系统配置校验、基础数据导入等一次性启动任务。



```

/**
 * 数据库初始化Runner
 * 支持通过命令行参数--init-data触发初始化
 */
@Component
@Order(1) // 标记执行顺序，数字越小越先执行
public class DatabaseInitCommandLineRunner implements CommandLineRunner {

    @Resource
    private DataInitService dataInitService;

    @Override
    public void run(String... args) throws Exception {
        log.info("开始执行数据库初始化任务...");

        // 判断是否传入初始化参数
        if (shouldInitializeData(args)) {
            dataInitService.initializeDefaultData(); // 初始化默认数据（如字典、权限）
            log.info("数据库初始化完成");
        } else {
            log.info("未检测到--init-data参数，跳过数据库初始化");
        }
    }

    /**
     * 判断是否需要初始化数据
     * @param args 命令行参数数组
     * @return true-需要初始化，false-跳过
     */
    private boolean shouldInitializeData(String[] args) {
        return Arrays.asList(args).contains("--init-data");
    }
}

/**
 * 系统健康检查Runner
 * 执行数据库、Redis等核心组件连通性检查
 */
@Component
@Order(2) // 晚于数据库初始化执行
public class HealthCheckCommandLineRunner implements CommandLineRunner {

```

```
@Override
public void run(String... args) throws Exception {
    log.info("开始执行系统健康检查...");

    checkDatabaseConnection(); // 数据库连接检查
    checkRedisConnection(); // Redis连接检查

    log.info("系统健康检查完成");
}

private void checkDatabaseConnection() {
    // 数据库连接校验逻辑
}

private void checkRedisConnection() {
    // Redis连接校验逻辑
}
}
```

| 02.3 增强型启动任务：ApplicationRunner

扩展点说明： `ApplicationRunner` 与 `CommandLineRunner` 功能类似，核心区别是提供 `ApplicationArguments` 对象，支持更灵活的命令行参数解析（区分选项参数/非选项参数）。

适用场景： 需要复杂参数处理的启动任务（如根据不同环境参数加载不同配置）。



```

/**
 * 系统配置加载Runner
 * 支持解析--profile等选项参数，加载对应环境配置
 */
@Component
public class SystemConfigApplicationRunner implements ApplicationRunner {

    @Override
    public void run(ApplicationArguments args) throws Exception {
        log.info("开始加载系统配置...");

        // 解析选项参数：--profile=dev/test/prod
        if (args.containsOption("profile")) {
            String profile = args.getOptionValues("profile").get(0);
            log.info("当前加载环境配置：{}", profile);
            // 加载对应环境的配置文件
        }

        // 解析非选项参数（无--前缀的参数）
        List<String> nonOptionArgs = args.getNonOptionArgs();
        if (!nonOptionArgs.isEmpty()) {
            log.info("接收到非选项参数：{}", nonOptionArgs);
        }

        loadSystemConfiguration(); // 加载核心配置
        log.info("系统配置加载完成");
    }

    /**
     * 加载系统核心配置
     */
    private void loadSystemConfiguration() {
        // 加载配置文件、初始化配置中心连接等逻辑
    }
}

```

02.4 Bean生命周期扩展：BeanPostProcessor

扩展点说明： **BeanPostProcessor** 是Spring框架核心扩展点，可在Bean初始化前后修改/包装Bean实例。

Spring的AOP、@Autowired注入等核心功能均基于此实现。

适用场景：统一实现AOP切面、Bean属性注入、性能监控、缓存代理、Bean合法性校验等。



```

/**
 * 自定义Bean后置处理器
 * 1. 为标记@PerformanceMonitor的Bean添加性能监控代理
 * 2. 为所有Service结尾的Bean添加缓存代理
 */
@Component
public class CustomBeanPostProcessor implements BeanPostProcessor {

    /**
     * Bean初始化前执行：此处为标记@PerformanceMonitor的Bean添加性能监控
     * @param bean 待处理的Bean实例
     * @param beanName Bean名称
     * @return 处理后的Bean（可返回代理对象）
     */
    @Override
    public Object postProcessBeforeInitialization(Object bean, String beanName) throws Exception {
        // 判断Bean是否标记性能监控注解
        if (bean.getClass().isAnnotationPresent(PerformanceMonitor.class)) {
            log.info("为Bean [{}] 初始化性能监控代理", beanName);
            return createPerformanceProxy(bean); // 创建性能监控代理
        }
        return bean; // 未标记则返回原Bean
    }

    /**
     * Bean初始化后执行：此处为Service Bean添加缓存代理
     * @param bean 初始化完成的Bean实例
     * @param beanName Bean名称
     * @return 处理后的Bean（可返回代理对象）
     */
    @Override
    public Object postProcessAfterInitialization(Object bean, String beanName) throws Exception {
        // 判断是否为Service结尾的Bean
        if (bean.getClass().getSimpleName().endsWith("Service")) {
            log.info("为Service Bean [{}] 初始化缓存代理", beanName);
            return createCacheProxy(bean); // 创建缓存代理
        }
        return bean; // 非Service则返回原Bean
    }

    /**
     * 创建性能监控代理：统计方法执行耗时

```

```

    * @param target 目标Bean实例
    * @return 带性能监控的代理对象
    */
    private Object createPerformanceProxy(Object target) {
        return Proxy.newProxyInstance(
            target.getClass().getClassLoader(), // 类加载器
            target.getClass().getInterfaces(), // 目标Bean实现的接口
            (proxy, method, args) -> { // 代理逻辑
                long startTime = System.currentTimeMillis(); // 记录开始时间
                try {
                    Object result = method.invoke(target, args); // 执行原方法
                    long endTime = System.currentTimeMillis(); // 记录结束时间
                    log.info("方法 [{}] 执行耗时: {}ms", method.getName(), endTime - s
                    return result;
                } catch (Exception e) {
                    log.error("方法 [{}] 执行异常", method.getName(), e);
                    throw e; // 抛出异常，不吞异常
                }
            }
        );
    }
}

/**
 * 创建缓存代理（示例）：为Service方法添加缓存逻辑
 * @param target 目标Bean实例
 * @return 带缓存的代理对象
 */
private Object createCacheProxy(Object target) {
    // 此处可实现缓存代理逻辑，如读取方法返回值并缓存
    return target;
}
}

// 性能监控注解：标记需要监控的Bean
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
public @interface PerformanceMonitor {
}

// 示例Service：演示代理效果
@PerformanceMonitor // 标记性能监控
public class OrderService {
    . . .
    . . .
    . . .
    . . .
    . . .
    . . .

```

```
    public void createOrder(Order order) {  
        // 订单创建核心逻辑  
    }  
}
```

| 02.5 Bean生命周期钩子：InitializingBean & DisposableBean

扩展点说明：

- **InitializingBean**：Bean所有属性注入完成后执行 **afterPropertiesSet()**，用于初始化资源；
- **DisposableBean**：Bean销毁前执行 **destroy()**，用于清理资源（如线程池、连接池）。

适用场景：连接池管理、定时任务启停、文件句柄释放、网络连接关闭等。



```

/**
 * 连接池管理器：演示Bean生命周期钩子使用
 * 初始化时创建线程池，销毁时优雅关闭
 */
@Component
public class ConnectionPoolManager implements InitializingBean, DisposableBean {

    // 业务线程池
    private ExecutorService executorService;
    // 定时任务线程池
    private ScheduledExecutorService scheduledExecutor;

    /**
     * Bean属性注入完成后执行：初始化线程池
     */
    @Override
    public void afterPropertiesSet() throws Exception {
        log.info("初始化连接池管理器...");

        // 创建固定大小业务线程池
        executorService = Executors.newFixedThreadPool(10);
        // 创建定时任务线程池
        scheduledExecutor = Executors.newScheduledThreadPool(5);

        startScheduledTasks(); // 启动定时任务

        log.info("连接池管理器初始化完成");
    }

    /**
     * Bean销毁前执行：优雅关闭线程池
     */
    @Override
    public void destroy() throws Exception {
        log.info("开始销毁连接池管理器...");

        // 关闭业务线程池（等待30秒，超时则强制关闭）
        if (executorService != null) {
            executorService.shutdown();
            if (!executorService.awaitTermination(30, TimeUnit.SECONDS)) {
                executorService.shutdownNow();
            }
        }
    }
}

```

```

    }

    // 关闭定时任务线程池
    if (scheduledExecutor != null) {
        scheduledExecutor.shutdown();
        if (!scheduledExecutor.awaitTermination(30, TimeUnit.SECONDS)) {
            scheduledExecutor.shutdownNow();
        }
    }

    log.info("连接池管理器销毁完成");
}

/**
 * 启动定时清理任务
 */
private void startScheduledTasks() {
    // 每小时执行一次缓存清理
    scheduledExecutor.scheduleAtFixedRate(() -> {
        log.info("执行定时缓存清理任务...");
        // 清理过期缓存、无效连接等逻辑
    }, 0, 1, TimeUnit.HOURS);
}
}

```

02.6 事件驱动扩展：ApplicationEvent & @EventListener

扩展点说明：基于发布-订阅模式，通过 `ApplicationEventPublisher` 发布自定义事件，`@EventListener` 注解监听并处理事件，彻底解耦业务逻辑。

适用场景：订单创建后的库存扣减、积分发放、消息通知等后续操作。



```

/**
 * 订单创建事件：自定义业务事件
 * 携带订单、用户信息，供监听器使用
 */
public class OrderCreatedEvent extends ApplicationEvent {
    private final Order order; // 订单信息
    private final User user;   // 用户信息

    // 构造函数：必须传入事件源（通常是发布者自身）
    public OrderCreatedEvent(Object source, Order order, User user) {
        super(source);
        this.order = order;
        this.user = user;
    }

    // Getter方法：供监听器获取数据
    public Order getOrder() { return order; }
    public User getUser() { return user; }
}

/**
 * 订单服务：事件发布者
 * 订单创建完成后发布事件，不耦合后续处理逻辑
 */
@Service
public class OrderService {

    // 注入事件发布者
    @Resource
    private ApplicationEventPublisher eventPublisher;

    /**
     * 创建订单：核心业务逻辑
     * @param request 订单创建请求
     * @return 生成的订单
     */
    public Order createOrder(CreateOrderRequest request) {
        // 1. 构建订单对象
        Order order = buildOrder(request);
        // 2. 保存订单到数据库
        orderRepository.save(order);
        // 3. 发布订单创建事件
    }
}

```

```

        eventPublisher.publishEvent(new OrderCreatedEvent(this, order, request.getUser());
        // 4. 返回订单
        return order;
    }

    // 构建订单对象（示例）
    private Order buildOrder(CreateOrderRequest request) {
        Order order = new Order();
        // 填充订单信息（用户ID、商品、金额等）
        return order;
    }
}

/**
 * 库存监听器：处理订单创建事件，扣减商品库存
 */
@Component
public class InventoryEventListener {

    @Resource
    private InventoryService inventoryService;

    /**
     * 监听订单创建事件，扣减库存
     * @param event 订单创建事件
     */
    @EventListener
    public void handleOrderCreated(OrderCreatedEvent event) {
        Order order = event.getOrder();
        log.info("订单创建事件：开始扣减库存，订单ID: {}", order.getId());

        try {
            // 遍历订单商品，逐个扣减库存
            order.getItems().forEach(item -> {
                inventoryService.reduceStock(item.getProductId(), item.getQuantity());
            });
            log.info("库存扣减完成，订单ID: {}", order.getId());
        } catch (Exception e) {
            log.error("库存扣减失败，订单ID: {}", order.getId(), e);
            // 可添加补偿逻辑（如重试、记录异常订单）
        }
    }
}

```



```

}

/**
 * 积分监听器：处理订单创建事件，发放用户积分
 */
@Component
public class PointsEventListener {

    @Resource
    private PointsService pointsService;

    /**
     * 监听订单创建事件，计算并发放积分
     * @param event 订单创建事件
     */
    @EventListener
    public void handleOrderCreated(OrderCreatedEvent event) {
        Order order = event.getOrder();
        User user = event.getUser();
        log.info("订单创建事件：开始计算积分，订单ID: {}", order.getId());

        try {
            // 计算积分：订单金额*0.01（示例规则）
            int points = (int) (order.getTotalAmount().doubleValue() * 0.01);
            // 发放积分
            pointsService.addPoints(user.getId(), points, "订单" + order.getId() + "当
            log.info("积分增加完成，用户ID: {}", user.getId(), points);
        } catch (Exception e) {
            log.error("积分增加失败，订单ID: {}", order.getId(), e);
        }
    }
}

/**
 * 通知监听器：处理订单创建事件，发送邮件/短信通知
 */
@Component
public class NotificationEventListener {

    @Resource
    private EmailService emailService;
    @Resource
    private SmsService smsService:

```

```

/**
 * 监听订单创建事件，发送订单确认通知
 * @param event 订单创建事件
 */
@EventListener
public void handleOrderCreated(OrderCreatedEvent event) {
    Order order = event.getOrder();
    User user = event.getUser();
    log.info("订单创建事件：发送通知，订单ID: {}", order.getId());

    try {
        // 发送邮件通知
        emailService.sendOrderConfirmation(user.getEmail(), order);
        // 发送短信通知
        smsService.sendOrderConfirmation(user.getPhone(), order);
        log.info("订单通知发送完成，订单ID: {}", order.getId());
    } catch (Exception e) {
        log.error("订单通知发送失败，订单ID: {}", order.getId(), e);
    }
}
}

```

02.7 高级事件监听：条件监听 & 事务监听

扩展点说明：

- 条件监听：只处理满足特定条件的事件（如VIP用户订单、大额订单）；
- 事务监听：在事务提交/回滚后执行（通过 `@TransactionalEventListener`），保证数据一致性。

适用场景： VIP用户特殊处理、大额订单风控、事务成功后通知外部系统、事务失败后补偿。



```

/**
 * 条件事件监听器：针对不同类型订单做差异化处理
 */
@Component
public class ConditionalEventListener {

    /**
     * 处理VIP用户订单：仅当用户等级≥3时触发
     * @param event 订单创建事件
     */
    @EventListener(condition = "#event.user.vipLevel >= 3")
    public void handleVipOrderCreated(OrderCreatedEvent event) {
        User user = event.getUser();
        Order order = event.getOrder();
        log.info("VIP用户订单处理，用户等级：{}", 订单ID：{}", user.getVipLevel(), order.

        // VIP专属逻辑：如优先发货、赠送优惠券等
        processVipOrder(order);
    }

    /**
     * 处理大额订单：仅当订单金额≥10000时触发
     * @param event 订单创建事件
     */
    @EventListener(condition = "#event.order.totalAmount >= 10000")
    public void handleLargeOrderCreated(OrderCreatedEvent event) {
        Order order = event.getOrder();
        log.info("大额订单处理，订单金额：{}", 订单ID：{}", order.getTotalAmount(), order

        // 大额订单逻辑：如风控审核、人工跟进等
        processLargeOrder(order);
    }

    // VIP订单处理逻辑
    private void processVipOrder(Order order) { /* 业务逻辑 */ }
    // 大额订单处理逻辑
    private void processLargeOrder(Order order) { /* 业务逻辑 */ }
}

/**
 * 事务事件监听器：保证事件处理与订单事务一致
 */

```

```

@Component
public class TransactionalEventListener {

    /**
     * 订单事务提交后执行：通知外部系统（如物流、财务）
     * @param event 订单创建事件
     */
    @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
    public void handleOrderCreatedAfterCommit(OrderCreatedEvent event) {
        Order order = event.getOrder();
        log.info("订单事务提交后处理，订单ID: {}", order.getId());

        // 通知外部系统：仅当订单事务提交成功后执行
        sendOrderCreatedNotificationToExternalSystem(order);
    }

    /**
     * 订单事务回滚后执行：补偿处理（如恢复库存、记录异常）
     * @param event 订单创建事件
     */
    @TransactionalEventListener(phase = TransactionPhase.AFTER_ROLLBACK)
    public void handleOrderCreatedAfterRollback(OrderCreatedEvent event) {
        Order order = event.getOrder();
        log.info("订单事务回滚后处理，订单ID: {}", order.getId());

        // 补偿逻辑：如恢复扣减的库存、通知用户订单创建失败
        compensateForFailedOrder(order);
    }

    // 通知外部系统逻辑
    private void sendOrderCreatedNotificationToExternalSystem(Order order) { /* 业务逻辑 */ }

    // 订单失败补偿逻辑
    private void compensateForFailedOrder(Order order) { /* 业务逻辑 */ }
}

```

02.8 Web层扩展：HandlerInterceptor

扩展点说明：Spring MVC提供的请求拦截器，支持在请求处理前、处理后、完成后三个阶段插入逻辑，可注入Spring Bean，适配Web场景。

适用场景：权限认证、请求日志、性能监控、参数校验、响应数据统一封装等。



```

/**
 * 请求日志拦截器：记录请求全生命周期信息
 */
@Component
public class RequestLoggingInterceptor implements HandlerInterceptor {
    // 请求开始时间标识
    private static final String START_TIME = "startTime";
    // 请求ID标识（用于链路追踪）
    private static final String REQUEST_ID = "requestId";

    /**
     * 请求处理前执行：初始化请求ID、记录开始时间
     * @return true-放行，false-拦截
     */
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response) {
        // 生成唯一请求ID（链路追踪）
        String requestId = UUID.randomUUID().toString();
        // 记录请求开始时间
        long startTime = System.currentTimeMillis();

        // 存入request属性，供后续阶段使用
        request.setAttribute(REQUEST_ID, requestId);
        request.setAttribute(START_TIME, startTime);
        // 存入MDC，方便日志打印
        MDC.put("requestId", requestId);

        // 打印请求基本信息
        log.info("请求开始 - URL: {}, Method: {}, IP: {}, User-Agent: {}",
            request.getRequestURL(),
            request.getMethod(),
            getClientIpAddress(request),
            request.getHeader("User-Agent"));

        return true; // 放行请求
    }

    /**
     * 请求处理后（视图渲染前）执行：记录处理耗时
     */
    @Override
    public void postHandle(HttpServletRequest request, HttpServletResponse response,

```

```

        long startTime = (Long) request.getAttribute(START_TIME);
        long endTime = System.currentTimeMillis();

        // 打印请求处理耗时
        log.info("请求处理完成 - 耗时: {}ms, Status: {}",
                endTime - startTime,
                response.getStatus());
    }

    /**
     * 请求完全结束后执行：清理资源、打印结束日志
     */
    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse response, Object ex) throws Exception {
        // 如有异常，打印异常日志
        if (ex != null) {
            log.error("请求处理异常", ex);
        }
        // 清理MDC，避免线程复用导致数据污染
        MDC.clear();
        log.info("请求结束 - RequestID: {}", request.getAttribute(REQUEST_ID));
    }

    /**
     * 获取客户端真实IP（处理反向代理场景）
     * @param request HTTP请求
     * @return 客户端IP
     */
    private String getClientIpAddress(HttpServletRequest request) {
        // 优先取X-Forwarded-For（反向代理如Nginx）
        String xForwardedFor = request.getHeader("X-Forwarded-For");
        if (xForwardedFor != null && !xForwardedFor.isEmpty()) {
            return xForwardedFor.split(",")[0].trim();
        }
        // 其次取X-Real-IP
        String xRealIp = request.getHeader("X-Real-IP");
        if (xRealIp != null && !xRealIp.isEmpty()) {
            return xRealIp;
        }
        // 最后取原生IP
        return request.getRemoteAddr();
    }

```

```

}

/**
 * 权限认证拦截器：校验用户Token有效性
 */
@Component
public class AuthenticationInterceptor implements HandlerInterceptor {

    @Resource
    private JwtTokenUtil jwtTokenUtil;

    /**
     * 请求处理前执行：校验Token
     */
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response) {
        // 跳过带@NoAuth注解的方法/类
        if (handler instanceof HandlerMethod) {
            HandlerMethod handlerMethod = (HandlerMethod) handler;
            if (handlerMethod.hasMethodAnnotation(NoAuth.class) ||
                handlerMethod.getBeanType().isAnnotationPresent(NoAuth.class)) {
                return true;
            }
        }

        // 获取Token（格式：Bearer xxx）
        String token = request.getHeader("Authorization");
        if (token == null || !token.startsWith("Bearer ")) {
            // Token为空，返回401未授权
            response.setStatus(HttpStatus.UNAUTHORIZED.value());
            response.getWriter().write("{\"code\":401,\"msg\":\"未授权访问，Token为空\"}");
            return false;
        }

        // 截取Token（去掉Bearer前缀）
        token = token.substring(7);

        try {
            // 校验Token并获取用户ID
            String userId = jwtTokenUtil.getUserIdFromToken(token);
            if (userId != null) {
                // 存入request，供后续接口使用
                request.setAttribute("userId", userId);
                return true;
            }
        } catch (Exception e) {
            // Token无效，返回401未授权
            response.setStatus(HttpStatus.UNAUTHORIZED.value());
            response.getWriter().write("{\"code\":401,\"msg\":\"Token无效\"}");
            return false;
        }
    }
}

```



```

        }
    } catch (Exception e) {
        log.warn("Token验证失败", e);
    }

    // Token无效, 返回401未授权
    response.setStatus(HttpStatus.UNAUTHORIZED.value());
    response.getWriter().write("{\"code\":401,\"msg\":\"Token无效或已过期\"}");
    return false;
}
}

/**
 * Web配置类: 注册拦截器, 指定拦截/放行规则
 */
@Configuration
public class WebConfig implements WebMvcConfigurer {

    @Resource
    private RequestLoggingInterceptor requestLoggingInterceptor;
    @Resource
    private AuthenticationInterceptor authenticationInterceptor;

    /**
     * 注册拦截器
     */
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        // 日志拦截器: 拦截所有请求
        registry.addInterceptor(requestLoggingInterceptor)
            .addPathPatterns("/**");

        // 认证拦截器: 仅拦截/api/**, 放行登录/注册等公开接口
        registry.addInterceptor(authenticationInterceptor)
            .addPathPatterns("/api/**")
            .excludePathPatterns("/api/login", "/api/register", "/api/public/**");
    }
}

// 无需认证注解: 标记公开接口
@Target({ElementType.METHOD, ElementType.TYPE})
@Retention(RetentionPolicy.RUNTIME)

```

```
public @interface NoAuth {  
}
```

| 02.9 底层请求过滤：Filter

扩展点说明：Servlet规范的过滤器，执行时机早于Spring拦截器（请求到达Spring容器前），功能更底层，可处理静态资源请求。

适用场景：跨域处理、字符编码设置、请求体缓存、安全过滤（如XSS、SQL注入）等。



```

/**
 * 跨域过滤器：解决前端跨域请求问题
 */
@Component
public class CorsFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException, ServletException {
        HttpServletResponse httpResponse = (HttpServletResponse) response;
        HttpServletRequest httpRequest = (HttpServletRequest) request;

        // 设置跨域响应头
        httpResponse.setHeader("Access-Control-Allow-Origin", "*"); // 允许所有源（生产环境需谨慎）
        httpResponse.setHeader("Access-Control-Allow-Methods", "GET, POST, PUT, DELETE, OPTIONS");
        httpResponse.setHeader("Access-Control-Allow-Headers", "Content-Type, Authorization");
        httpResponse.setHeader("Access-Control-Max-Age", "3600"); // 预检请求缓存时间（单位：秒）

        // 处理OPTIONS预检请求：直接返回200
        if ("OPTIONS".equals(httpRequest.getMethod())) {
            httpResponse.setStatus(HttpServletResponse.SC_OK);
            return;
        }

        // 放行请求到下一个过滤器/Spring容器
        chain.doFilter(request, response);
    }
}

```

```

/**
 * 请求体缓存过滤器：解决POST/PUT请求体只能读取一次的问题
 */
@Component
public class RequestWrapperFilter implements Filter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain) throws IOException, ServletException {
        // 仅处理POST/PUT请求
        if (request instanceof HttpServletRequest) {
            HttpServletRequest httpRequest = (HttpServletRequest) request;
            String method = httpRequest.getMethod();
            if ("POST".equals(method) || "PUT".equals(method)) {
                // 包装请求，缓存请求体
            }
        }
        chain.doFilter(request, response);
    }
}

```

```

        CachedBodyHttpServletRequest wrappedRequest = new CachedBodyHttpServ:
        chain.doFilter(wrappedRequest, response);
        return;
    }
}
// 其他请求直接放行
chain.doFilter(request, response);
}
}

/**
 * 缓存请求体的HttpServletRequest包装类
 * 解决请求体InputStream只能读取一次的问题
 */
public class CachedBodyHttpServletRequest extends HttpServletRequestWrapper {
    // 缓存的请求体字节数组
    private byte[] cachedBody;

    /**
     * 构造函数：读取并缓存请求体
     * @param request 原始请求
     */
    public CachedBodyHttpServletRequest(HttpServletRequest request) throws IOException {
        super(request);
        // 将请求体读取为字节数组并缓存
        this.cachedBody = StreamUtils.copyToByteArray(request.getInputStream());
    }

    /**
     * 重写getInputStream：返回缓存的请求体
     */
    @Override
    public ServletInputStream getInputStream() throws IOException {
        return new CachedBodyServletInputStream(this.cachedBody);
    }

    /**
     * 重写getReader：返回缓存的请求体字符流
     */
    @Override
    public BufferedReader getReader() throws IOException {
        ByteArrayInputStream byteArrayInputStream = new ByteArrayInputStream(this.cac

```

```

        return new BufferedReader(new InputStreamReader(byteArrayInputStream));
    }

    /**
     * 获取缓存的请求体字节数组
     * @return 请求体字节数组
     */
    public byte[] getCachedBody() {
        return cachedBody;
    }

    /**
     * 缓存请求体的ServletInputStream实现
     */
    private static class CachedBodyServletInputStream extends ServletInputStream {
        private final ByteArrayInputStream inputStream;

        public CachedBodyServletInputStream(byte[] cachedBody) {
            this.inputStream = new ByteArrayInputStream(cachedBody);
        }

        @Override
        public int read() throws IOException {
            return inputStream.read();
        }

        @Override
        public boolean isFinished() {
            return inputStream.available() == 0;
        }

        @Override
        public boolean isReady() {
            return true;
        }

        @Override
        public void setReadListener(ReadListener listener) {
            // 无需实现（非异步场景）
        }
    }
}

```

扩展点说明：基于 `@Conditional` 系列注解，实现“开箱即用”的模块化配置，根据类路径、配置属性等条件自动装配Bean。

适用场景：通用组件封装、第三方组件集成、企业内部标准化模块开发等。



```

/**
 * 缓存自动配置类：自定义Starter核心
 * 根据配置自动装配缓存管理器、缓存服务
 */
@Configuration
@EnableConfigurationProperties(CacheProperties.class) // 绑定配置属性
@ConditionalOnClass(RedisTemplate.class) // 存在RedisTemplate时才生效
@ConditionalOnProperty(prefix = "app.cache", name = "enabled", havingValue = "true",
public class CacheAutoConfiguration {

    /**
     * 装配自定义缓存管理器
     * @param properties 缓存配置属性
     * @return 缓存管理器
     */
    @Bean
    @ConditionalOnMissingBean // 未自定义时才装配
    public CacheManager customCacheManager(CacheProperties properties) {
        CustomCacheManager cacheManager = new CustomCacheManager();
        cacheManager.setTtl(properties.getTtl()); // 设置默认过期时间
        cacheManager.setPrefix(properties.getPrefix()); // 设置缓存Key前缀
        cacheManager.setMaxSize(properties.getMaxSize()); // 设置缓存最大容量
        return cacheManager;
    }

    /**
     * 装配缓存服务
     * @param cacheManager 缓存管理器
     * @return 缓存服务
     */
    @Bean
    @ConditionalOnBean(CacheManager.class) // 存在CacheManager时才生效
    public CacheService cacheService(CacheManager cacheManager) {
        return new CacheService(cacheManager);
    }
}

/**
 * 缓存配置属性类：绑定application.yml中的配置
 * 前缀： app.cache
 */
@ConfigurationProperties(prefix = "app.cache")

```

```

public class CacheProperties {
    // 是否启用缓存（默认开启）
    private boolean enabled = true;
    // 缓存默认过期时间（秒）
    private int ttl = 3600;
    // 缓存Key前缀
    private String prefix = "app:cache:";
    // 缓存最大容量
    private int maxSize = 1000;

    // Getter & Setter
    public boolean isEnabled() { return enabled; }
    public void setEnabled(boolean enabled) { this.enabled = enabled; }
    public int getTtl() { return ttl; }
    public void setTtl(int ttl) { this.ttl = ttl; }
    public String getPrefix() { return prefix; }
    public void setPrefix(String prefix) { this.prefix = prefix; }
    public int getMaxSize() { return maxSize; }
    public void setMaxSize(int maxSize) { this.maxSize = maxSize; }
}

```

```

/**

```

```

 * 缓存服务：封装缓存操作，对外提供统一API

```

```

 */

```

```

public class CacheService {
    // 注入缓存管理器
    private final CacheManager cacheManager;

    // 构造函数注入
    public CacheService(CacheManager cacheManager) {
        this.cacheManager = cacheManager;
    }

    /**
     * 存入缓存（使用默认配置）
     * @param key 缓存Key
     * @param value 缓存值
     */
    public void put(String key, Object value) {
        cacheManager.getCache("default").put(key, value);
    }
}

```

```

...

```



```

/**
 * 从缓存获取数据
 * @param key 缓存Key
 * @param type 返回值类型
 * @return 缓存值（null表示未命中）
 */
public <T> T get(String key, Class<T> type) {
    Cache.ValueWrapper wrapper = cacheManager.getCache("default").get(key);
    return wrapper != null ? type.cast(wrapper.get()) : null;
}
}

/**
 * 自定义缓存管理器（示例）
 */
public class CustomCacheManager implements CacheManager {
    private int ttl;
    private String prefix;
    private int maxSize;

    // Getter & Setter
    public int getTtl() { return ttl; }
    public void setTtl(int ttl) { this.ttl = ttl; }
    public String getPrefix() { return prefix; }
    public void setPrefix(String prefix) { this.prefix = prefix; }
    public int getMaxSize() { return maxSize; }
    public void setMaxSize(int maxSize) { this.maxSize = maxSize; }

    @Override
    public Cache getCache(String name) {
        // 实现缓存获取逻辑
        return null;
    }

    @Override
    public Collection<String> getCacheNames() {
        // 实现缓存名称获取逻辑
        return Collections.emptyList();
    }
}
}

```

02.11 环境适配: @Profile & @Conditional

扩展点说明：通过 `@Profile`（环境）、`@ConditionalOnProperty`（配置）等注解，实现不同环境/配置下的差异化Bean装配。

适用场景：开发/测试/生产环境配置隔离、功能开关、可选组件装配等。



```

/**
 * 环境差异化配置类
 * 根据不同环境装配不同的数据源、日志级别等
 */
@Configuration
public class EnvironmentSpecificConfiguration {

    /**
     * 开发环境配置：嵌入式数据库（H2）
     */
    @Configuration
    @Profile("dev") // 仅dev环境生效
    static class DevelopmentConfiguration {

        /**
         * 装配开发环境数据源
         * @return H2内存数据库
         */
        @Bean
        public DataSource devDataSource() {
            HikariDataSource dataSource = new HikariDataSource();
            dataSource.setJdbcUrl("jdbc:h2:mem:devdb"); // 内存数据库，重启丢失
            dataSource.setUsername("sa");
            dataSource.setPassword("");
            dataSource.setMaximumPoolSize(5); // 小连接池，节省资源
            return dataSource;
        }

        /**
         * 开发环境日志级别：DEBUG
         * @return 日志级别
         */
        @Bean
        public LogLevel devLogLevel() {
            return LogLevel.DEBUG;
        }
    }

    /**
     * 生产环境配置：MySQL数据库
     */
    @Configuration

```

```

@Profile("prod") // 仅prod环境生效
static class ProductionConfiguration {

    /**
     * 装配生产环境数据源
     * @param jdbcUrl 数据库URL（从配置文件读取）
     * @param username 数据库用户名
     * @param password 数据库密码
     * @return MySQL数据源
     */
    @Bean
    @ConfigurationProperties(prefix = "spring.datasource") // 绑定配置文件属性
    public DataSource prodDataSource(
        @Value("${spring.datasource.url}") String jdbcUrl,
        @Value("${spring.datasource.username}") String username,
        @Value("${spring.datasource.password}") String password) {
        HikariDataSource dataSource = new HikariDataSource();
        dataSource.setJdbcUrl(jdbcUrl);
        dataSource.setUsername(username);
        dataSource.setPassword(password);
        dataSource.setMaximumPoolSize(20); // 大连接池，支撑高并发
        dataSource.setConnectionTimeout(30000); // 连接超时时间
        return dataSource;
    }

    /**
     * 生产环境日志级别：INFO
     * @return 日志级别
     */
    @Bean
    public LogLevel prodLogLevel() {
        return LogLevel.INFO;
    }
}

/**
 * 通用异步任务执行器：所有环境共用
 * @return 线程池执行器
 */
@Bean
public AsyncTaskExecutor asyncTaskExecutor() {
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
    executor.setCorePoolSize(10);
    executor.setMaxPoolSize(20);
    executor.setQueueCapacity(100);
    executor.setThreadNamePrefix("async-task-executor-");
    executor.initialize();
    return executor;
}

```

```

        executor.setCorePoolSize(5); // 核心线程数
        executor.setMaxPoolSize(10); // 最大线程数
        executor.setQueueCapacity(100); // 队列容量
        executor.setThreadNamePrefix("async-"); // 线程名称前缀
        executor.initialize(); // 初始化线程池
        return executor;
    }

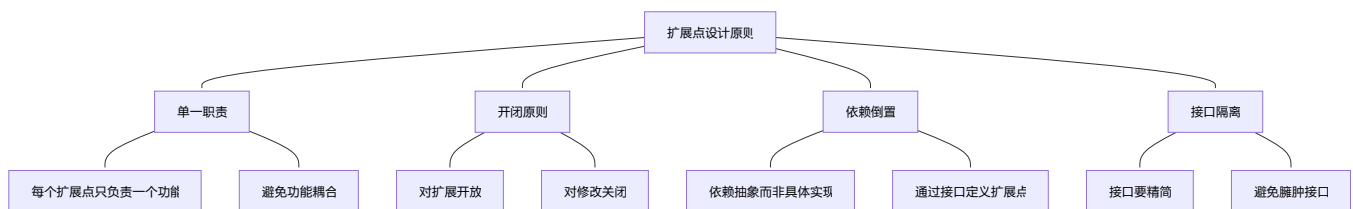
    /**
     * 默认邮件服务：所有环境共用
     * @return 邮件服务
     */
    @Bean
    @ConditionalOnMissingBean // 未自定义时装配默认实现
    public EmailService defaultEmailService() {
        return new DefaultEmailService();
    }
}

// 日志级别枚举（示例）
enum LogLevel {
    DEBUG, INFO, WARN, ERROR
}

```

03

扩展点设计原则：保证扩展性与可维护性



04

扩展点选型指南：按场景精准匹配

业务场景	推荐扩展点	核心原因
应用启动初始化	ApplicationListener	生命周期清晰， 支持监听启动完成事件
业务逻辑解耦	@EventListener	发布-订阅模式，异步解耦，便于扩展
跨切面功能（监控/ 缓存）	BeanPostProcessor	统一处理多个Bean， 实现全局横切逻辑
Web请求处理（权限/ 日志）	HandlerInterceptor	专为Spring MVC设计， 可注入Bean，适配Web场景
全局请求过滤（跨域/ 编码）	Filter	执行时机早，支持静态资源， 基于Servlet规范
环境差异化配置	@Profile + @Conditional	灵活的条件装配，适配多环境部署

05

扩展点最佳实践：避坑与性能优化

05.1 常见问题解决方案

问题原因	解决方案
事件监听器执行顺序不确定	使用 <code>@Order</code> 注解指定执行顺序（数字越小越先执行）
异步事件处理异常被忽略	配置 <code>AsyncUncaughtExceptionHandler</code> 捕获异步异常
扩展点过多导致启动缓慢	对非核心扩展点使用 <code>@Lazy</code> 延迟初始化
Bean循环依赖问题	通过 <code>@EventListener</code> 解耦，避免直接依赖

05.2 性能优化建议

- 1. 异步处理：事件监听、启动任务等非核心逻辑使用异步线程池，避免阻塞主流程；



```

/**
 * 异步事件配置：优化事件处理性能
 */
@Configuration
@EnableAsync // 开启异步支持
public class AsyncEventConfiguration {

    /**
     * 事件处理专用线程池
     * @return 线程池执行器
     */
    @Bean(name = "eventTaskExecutor")
    public TaskExecutor eventTaskExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5); // 核心线程数
        executor.setMaxPoolSize(20); // 最大线程数
        executor.setQueueCapacity(500); // 队列容量
        executor.setThreadNamePrefix("event-"); // 线程名称前缀
        // 拒绝策略：核心线程忙时，由调用线程执行（避免丢任务）
        executor.setRejectedExecutionHandler(new ThreadPoolExecutor.CallerRunsPolicy());
        executor.initialize();
        return executor;
    }

    /**
     * 异步异常处理器：捕获异步线程中的异常
     * @return 异常处理器
     */
    @Bean
    public AsyncUncaughtExceptionHandler asyncExceptionHandler() {
        return (ex, method, params) -> {
            log.error("异步方法 [{}] 执行异常", method.getName(), ex);
            // 可添加告警、重试等补偿逻辑
        };
    }
}

/**
 * 高性能事件监听器：使用异步线程池处理事件
 */
@Component
public class HighPerformanceEventListener {

```

```

// 注入事件专用线程池
@Resource(name = "eventTaskExecutor")
private TaskExecutor eventTaskExecutor;

/**
 * 异步处理订单事件：指定线程池
 * @param event 订单创建事件
 */
@EventListener
@Async("eventTaskExecutor") // 指定异步线程池
public void handleOrderEvent(OrderCreatedEvent event) {
    // 异步处理订单事件，不阻塞主线程
    processOrderEvent(event);
}

/**
 * 批量处理事件：提交到线程池执行
 * @param events 事件列表
 */
public void handleBatchEvent(List<OrderCreatedEvent> events) {
    eventTaskExecutor.execute(() -> {
        // 批量处理事件，提升吞吐量
        processBatchEvents(events);
    });
}

// 处理单个订单事件
private void processOrderEvent(OrderCreatedEvent event) { /* 业务逻辑 */ }
// 批量处理订单事件
private void processBatchEvents(List<OrderCreatedEvent> events) { /* 业务逻辑 */ }

```

2. 线程池配置：根据业务场景合理设置核心线程数、最大线程数、队列容量，避免线程耗尽；
3. 避免耗时操作：扩展点中不执行大文件读写、远程接口同步调用等耗时操作；
4. 异常处理：扩展点中必须捕获异常并记录日志，避免影响主流程。

05.3 测试策略

1. 单元测试：单独测试每个扩展点，使用 `@MockBean` 模拟依赖；

2. 顺序测试：验证多个扩展点的执行顺序是否符合预期；
3. 异常测试：模拟扩展点执行异常，验证是否不影响主流程。

06

总结：让SpringBoot应用“灵活应变”

SpringBoot的扩展点体系，本质是一套标准化的“插件化”开发框架。

从启动阶段的 `ApplicationListener`，到Bean生命周期的 `BeanPostProcessor`，再到Web层的 `HandlerInterceptor`、事件驱动的 `@EventListener`，每个扩展点都有其专属场景。

掌握这些扩展点，核心是理解“**对扩展开放、对修改关闭**”的设计思想——新增功能时，无需改动原有代码，只需开发对应的扩展组件，让应用像“乐高积木”一样灵活组装。

好的架构不是一蹴而就，而是能够优雅地适应变化。

用好SpringBoot扩展点，让你的代码从“硬编码”走向“插件化”，从容应对频繁的业务需求变更！

推荐文章：

[java面试100题讲解源文件](#)

[Spring Boot Starter 深度解析与实战系列](#)

[《MySQL系列教程》19篇](#)

